TÀI LIỆU TỔNG HỢP KỸ THUẬT

CHERRY CORE 2026

Hệ lõi trích xuất thông tin điều tra từ dữ liệu âm thanh tiếng Việt

Phạm vi: toàn bộ mã nguồn, tài liệu, script, model, prompt và output trong thư mục cherry_core

Ngày biên soạn: 13/03/2026

Nguồn tổng hợp: hồ sơ kế hoạch NCKH và khảo sát repo Cherry Core

Tóm tắt

Tài liệu này tổng hợp toàn bộ nền tảng kỹ thuật của Cherry Core - lõi xử lý âm thanh điều tra tiếng Việt được xây theo hướng offline-first, module hóa và sẵn sàng mở rộng sang hệ thống giao diện web phục vụ điều tra viên. Tài liệu không viết lại hồ sơ kế hoạch NCKH gốc, mà chuyển hóa kế hoạch đó thành một bức tranh kỹ thuật bám sát hiện trạng repo, qua đó cho thấy những thành phần đã có mã nguồn hoặc minh chứng chạy thử và những nhánh vẫn đang ở mức đề xuất nghiên cứu.

Trong phạm vi cherry_core, các khối kỹ thuật nổi bật gồm: nhận dạng tiếng nói tiếng Việt bằng Whisper V2, PhoWhisper và nhánh WhisperX; tiền xử lý và giảm ảo giác bằng Silero VAD, cấu hình giải mã bảo thủ, Bag-of-Hallucinations và delooping; speaker diarization bằng Pyannote hoặc SpeechBrain ECAPA-TDNN kết hợp VBx; hiệu chỉnh transcript bằng rule-based phonetic correction, ProtonX và LLM cục bộ; cuối cùng là lớp phân tích forensic dựa trên prompt module, scenario YAML và LLM cục bộ llama.cpp hoặc vLLM.

Các mục tiêu nghiên cứu trong kế hoạch NCKH gốc được tái sử dụng như mốc định hướng kỹ thuật:

- (1) Nghiên cứu, đánh giá và triển khai các kỹ thuật giảm hallucination trong mô hình ASR tiếng Việt, đặc biệt trên dữ liệu âm thanh chất lượng thấp, có tạp âm và giọng vùng miền.
- (2) Xây dựng pipeline nhận dạng tiếng nói (ASR) và phân định người nói (Speaker Diarization) hoàn toàn offline, đảm bảo an toàn thông tin.
- (3) Phát triển hệ thống khai phá thông tin tình báo tự động từ bản phiên âm sử dụng mô hình ngôn ngữ lớn (LLM) với kỹ thuật Chain-of-Verification nhằm đảm bảo độ tin cậy của kết quả.
- (4) Đánh giá và kiểm chứng toàn bộ hệ thống trên dữ liệu thực tế, so sánh với các giải pháp hiện có.

Bên cạnh phần mô tả hệ lõi, tài liệu dành riêng một chương để đặc tả giao diện web hoàn thiện dưới góc nhìn sản phẩm kỹ thuật. Chương này không khóa framework frontend cụ

thể, nhưng chuẩn hóa màn hình, DTO, endpoint v1, nguyên tắc lưu trữ offline và cách bọc các use case hiện có của Cherry Core thành API phục vụ GUI.

Danh mục viết tắt

Từ viết tắt	Giải nghĩa
ASR	Automatic Speech Recognition - nhận dạng tiếng nói tự động
VAD	Voice Activity Detection - phát hiện đoạn có tiếng nói
LLM	Large Language Model - mô hình ngôn ngữ lớn
WER	Word Error Rate - tỷ lệ lỗi từ trong ASR
DER	Diarization Error Rate - tỷ lệ lỗi phân định người nói
JSON	JavaScript Object Notation - định dạng dữ liệu có cấu trúc
VBx	Variational Bayes / Viterbi-based resegmentation cho diarization
CoVe	Chain-of-Verification - chuỗi xác minh nhiều bước cho LLM
GBNF	Grammar-Based Normal Form dùng để ràng buộc sinh văn bản
DTO	Data Transfer Object - đối tượng truyền dữ liệu giữa các lớp
CLI	Command Line Interface - giao diện dòng lệnh
GUI	Graphical User Interface - giao diện đồ họa người dùng

Chương 1. Bối cảnh, mục tiêu và phạm vi khảo sát

Bài toán cốt lõi của Cherry Core là khai thác thông tin từ âm thanh tiếng Việt trong bối cảnh điều tra và trinh sát kỹ thuật. Không giống các sản phẩm chuyển giọng nói sang văn bản thông thường, một hệ thống phục vụ điều tra phải đồng thời thỏa mãn bốn yêu cầu: độ chính xác ASR đủ cao trên dữ liệu nhiễu; khả năng phân biệt người nói; cơ chế kiểm soát ảo giác của cả ASR lẫn LLM; và vận hành hoàn toàn offline để bảo vệ dữ liệu nhạy cảm.

Hồ sơ kế hoạch NCKH gốc đặt ra các câu hỏi nghiên cứu xoay quanh giảm hallucination, speaker diarization, khai phá thông tin tình báo bằng LLM và kiến trúc pipeline offline. Trong tài liệu này, các câu hỏi đó được diễn giải lại dưới góc nhìn kỹ thuật triển khai của cherry_core, tức là tập trung vào những gì lõi hệ thống đang làm, đang thiếu và cần hoàn thiện.

Các câu hỏi nghiên cứu chính được kế thừa trực tiếp từ kế hoạch nguồn:

- - RQ1: Làm thế nào giảm thiểu hallucination trong mô hình ASR/NLP đối với tiếng Việt có giọng vùng miền, tạp âm và chất lượng âm thanh kém?
- - RQ2: Làm thế nào nhận dạng và phân biệt nhiều người nói (speaker diarization) trong hội thoại tiếng Việt với độ chính xác cao, không phụ thuộc hạ tầng cloud?
- - RQ3: Làm thế nào tự động khai phá thông tin tình báo từ bản phiên âm: đối tượng, sự kiện, mối liên hệ, mức độ nguy hiểm?
- - RQ4: Làm thế nào xây dựng pipeline trinh sát kỹ thuật hoàn chỉnh chạy offline, bảo mật, có thể triển khai trong môi trường không có internet?

Phạm vi khảo sát chỉ gồm cherry_core, bao gồm các thư mục core, application, infrastructure, prompts, scripts, tests, models, docs và output. Dự án vietnamese-stt-project cùng các tài liệu cũ ở root workspace bị loại khỏi tài liệu này để tránh pha trộn giữa phần lõi nghiên cứu và một nhánh ứng dụng web khác.

Phương pháp tổng hợp sử dụng bốn bước: đọc tài liệu kế hoạch NCKH do người dùng cung cấp; khảo sát mã nguồn để xác định port, adapter, entity, use case và script; đối chiếu artifact trong output và các báo cáo nghiên cứu trong docs; phân loại từng kỹ thuật

theo bốn nhãn trạng thái nhằm giữ tính trung thực giữa nội dung đã triển khai và nội dung mới ở mức đề xuất.

1.1. Nguyên tắc phân loại trạng thái kỹ thuật

Nhãn	Ý nghĩa
đã có code	Mã nguồn hoặc adapter đã hiện diện trong lõi hệ thống.
đã có script kiểm thử	Có script kiểm thử hoặc probe xác minh riêng.
đã có output minh chứng	Có artifact đầu ra hoặc báo cáo chạy thử trong repo.
mới ở mức adapter/probe	Mới dừng ở adapter, tài liệu nghiên cứu hoặc bước chuẩn bị.

1.2. Kết quả khảo sát tổng quát

- Cherry Core theo kiến trúc Hexagonal, ưu tiên tách domain khỏi adapter model.
- Lõi hiện có nhiều nhánh ASR và diarization song song, cho thấy dự án đang ở trạng thái nghiên cứu so sánh.
- Các thành phần correction và forensic prompt đã phong phú hơn lớp API hoặc UI; giao diện đồ họa chưa được xây ở mức sản phẩm.
- Một số điểm tích hợp còn lệch chuẩn interface, vì vậy tài liệu phải phân biệt rõ phần chạy được với phần còn cần ổn định hóa.

Chương 2. Kiến trúc lõi Cherry Core

Cherry Core được tổ chức theo mô hình Hexagonal Architecture, trong đó phần domain giữ vai trò trung tâm và mọi adapter mô hình AI chỉ đi qua port interface. Cách tổ chức này phù hợp với bối cảnh nghiên cứu vì cho phép thay thế backend ASR, diarization hoặc LLM mà không làm vỡ toàn bộ luồng nghiệp vụ.

2.1. Các lớp kiến trúc

Lớp	Thành phần tiêu biểu	Vai trò
Domain	core/domain/entities.py	Định nghĩa Transcript, SpeakerSegment, StrategicReport.
Ports	core/ports/*.py	Chuẩn hóa giao tiếp của ASR, diarization, correction và LLM.
Application	application/services, application/use_cases	Điều phối chép lời, correction, prompt render và phân tích.
Infrastructure	infrastructure/adapters, infrastructure/factories	Kết nối model thực với port interface, quản lý factory.
Presentation	presentation/cli/main.py	Điểm vào CLI hiện đại cho người vận hành kỹ thuật.
Artifacts	models/, output/, docs/	Lưu model offline, đầu ra chạy thử và báo cáo nghiên cứu.

2.2. Entity và port lõi

Nhóm	Tên	Vai trò kỹ thuật
Entity	Transcript	Lưu text, segment và metadata của ASR.
Entity	SpeakerSegment	Biểu diễn đoạn lời nói gắn speaker và timestamp.
Entity	StrategicReport	Báo cáo tình báo chiến lược hoặc tác chiến sau phân tích

		LLM.
Port	ITranscriber	Hợp đồng audio -> transcript.
Port	ISpeakerDiarizer	Hợp đồng audio -> speaker segments.
Port	ITextCorrector	Hợp đồng hiệu chỉnh transcript.
Port	ILLMEngine	Hợp đồng nạp model và sinh văn bản cho tác vụ phân tích.

2.3. Luồng dữ liệu lõi

- Audio đầu vào đi vào ASR adapter thông qua ITranscriber.
- Nếu bật correction, TranscribeAudioUseCase chạy thêm ITextCorrector hoặc service correction nhiều tầng.
- Nếu cần speaker-aware transcript, audio đi qua ISpeakerDiarizer, sau đó AlignmentService căn chỉnh speaker với transcript.
- PromptManager nạp template Jinja2 và scenario YAML để dựng prompt forensic.
- LLM adapter sinh báo cáo, sau đó ứng dụng hoặc CLI serialize báo cáo về JSON và text file trong output.
- Toàn bộ model được nạp từ models theo cấu hình cục bộ, không phụ thuộc cloud trong pha vận hành.

2.4. Quản lý model offline và artifact

Thư mục models chứa đầy đủ model cho ASR, correction, diarization, VAD và LLM. Điều này thể hiện nguyên tắc offline-first xuyên suốt: PhoWhisper, Whisper V2/V3, SpeechBrain, ProtonX, Silero, Vistral, Qwen3 và nhánh WhisperX đều có vị trí lưu cục bộ. Song song đó, output đóng vai trò evidence store, nơi lưu transcript thô, transcript đã hiệu chỉnh, speaker transcript và JSON report để phục vụ đánh giá kỹ thuật lẫn nghiệp vụ.

2.5. Điểm vào hiện tại của hệ

- presentation/cli/main.py: entry point tương đối hiện đại, dùng TranscribeAudioUseCase và GenerateStrategicReportUseCase.

- cli.py: entry point cũ hơn, cho thấy dấu vết refactor và một số import hoặc service không còn đồng bộ.
- Các script trong scripts: công cụ nghiên cứu, benchmark, setup và verify từng mô-đun.

Chương 3. Kỹ thuật đã triển khai hoặc đã có minh chứng trong Cherry Core

Chương này chỉ tập trung vào các kỹ thuật đã hiện diện trong mã nguồn hoặc đã có dấu vết kiểm thử, output, script probe trong repo. Mỗi kỹ thuật được gắn duy nhất một nhãn trạng thái nhằm tránh trình bày quá mức mức độ hoàn thiện.

Kỹ thuật	Nhãn trạng thái	Tóm tắt	Minh chứng tiêu biểu
Whisper V2 với cấu hình chống ảo giác	đã có output minh chứng	ASR nền hiện tại của Cherry Core, dùng cấu hình giải mã bảo thủ và bộ lọc lặp hậu xử lý.	infrastructure/adapters/asr/whisperv2_adapter.py; PROOF_RESULTS.md
PhoWhisper cho ASR tiếng Việt	đã có code	Adapter riêng cho PhoWhisper với model offline cục bộ, ưu tiên nhánh safetensors.	infrastructure/adapters/asr/phowhisper_adapter.py; models/phowhisper-safe/
WhisperX end-to-end (ASR + alignment + diarization)	đã có script kiểm thử	Nhánh nâng cấp tích hợp faster-whisper, word alignment và pyannote; đã có script setup và verify.	infrastructure/adapters/asr/whisperx_adapter.py; scripts/setup_whisperx_offline.py
Decoding thresholds giảm hallucination	đã có code	Các ngưỡng giải mã được hard-code trong WhisperV2Adapter để giảm nội dung bịa ở vùng im lặng.	infrastructure/adapters/asr/whisperv2_adapter.py

Silero VAD tiền xử lý	đã có code	VAD chạy offline bằng model JIT, ưu tiên bảo toàn thông tin với ngưỡng bảo thủ.	infrastructure/adapters/vad/silero_adapter.py; models/silero/
Bag-of-Hallucinations tiếng Việt và tiếng Anh	đã có code	Tập biểu thức gây ảo giác thường gặp được mã hóa trực tiếp trong HallucinationFilter.	infrastructure/adapters/asr/hallucination_filter.py
Delooping hậu xử lý	đã có code	Lớp khử lặp theo từ và theo cụm nhằm loại bỏ vòng lặp nhiều lần trong bản chép lời.	infrastructure/adapters/asr/hallucination_filter.py; infrastructure/adapters/asr/whisperv2_adapter.py
Pyannote diarization	mới ở mức adapter/probe	Adapter hỗ trợ Pyannote Community-1/3.1 nhưng vẫn phụ thuộc token Hugging Face và chưa có benchmark rõ trong repo.	infrastructure/adapters/diarization/pyannote_adapter.py; docs/CYCLE_2_RESEARCH_DIARIZATION.md
SpeechBrain ECAPA-TDNN diarization	đã có code	Nhánh diarization offline dựa trên embedding ECAPA-TDNN, clustering phổ và khối hợp nhất segment.	infrastructure/adapters/diarization/speechbrain_adapter.py; models/speechbrain/

VBx/Viterbi resegmentation	đã có code	Bộ làm mượt nhãn speaker dựa trên HMM/Viterbi để giảm speaker switching ngắn hạn.	infrastructure/adapters/diarization/vbx_refiner.py; infrastructure/adapters/diarization/speechbrain_adapter.py
Word-speaker alignment bằng IntervalTree	đã có code	Dịch vụ căn chỉnh timestamp word-level của ASR sang segment người nói, hỗ trợ tra cứu nhanh bằng IntervalTree.	core/services/alignment_service.py
Rule-based phonetic correction	đã có code	Bộ hiệu chỉnh lỗi đồng âm dị nghĩa tiếng Việt bằng từ điển JSON và biểu thức chính quy.	application/services/phonetic_corrector.py; assets/vocab/vietnamese_phonetic_errors.json
ProtonX Seq2Seq correction	đã có output minh chứng	Adapter hậu hiệu chỉnh câu chữ bằng mô hình ProtonX chạy local, hỗ trợ chia đoạn transcript dài.	infrastructure/adapters/correction/protonx_adapter.py; PROOF_RESULTS.md
LLM contextual correction	đã có code	CorrectionService kết hợp rule-based, vocabulary injection và LLM cục bộ để hiệu chỉnh theo ngữ	application/services/correction_service.py; application/services/phonetic_corrector.py

		cảnh miền.	
Prompt modules và scenario YAML	đã có script kiểm thử	Prompt điều tra được xây theo mô-đun Jinja2 và dữ liệu kịch bản YAML, đã có test render và test deep analysis.	application/services/prompt_manager.py; tests/test_prompt_renderer.py
JSON-only prompting và GBNF	đã có code	Prompt deep investigation ép đầu ra JSON và adapter llama.cpp có chỗ móc grammar, nhưng sample output cho thấy JSON chưa ổn định tuyệt đối.	prompts/templates/deep_investigation.j2; prompts/grammars/json_schema.gbnf
LLM offline qua llama.cpp	đã có output minh chứng	LlamaCppAdapter là backend LLM mặc định, dùng model GGUF và đã tạo được báo cáo phân tích mẫu trong output.	infrastructure/adapters/llm/llamacpp_adapter.py; output/Tiếp nhận yêu cầu đặt phòng của khách lẻ qua điện thoại_analysis_report.json
vLLM fallback qua WSL2/probe	đã có script kiểm thử	Adapter hiệu năng cao đã có, kèm script chuẩn bị WSL2 và tài liệu chu trình fallback về llama.cpp nếu Windows không phù hợp.	infrastructure/adapters/llm/vllm_adapter.py; scripts/setup_wsl_vllm.ps1

3.1. Cụm kỹ thuật ASR

Nhánh ASR của Cherry Core không đi theo một engine duy nhất. SystemFactory hiện chọn Whisper V2 làm champion mặc định để ưu tiên độ ổn định và giảm hallucination, trong khi PhoWhisper được giữ như một lựa chọn tăng cường tiếng Việt và WhisperX được phát triển như nhánh hợp nhất ASR + alignment + diarization.

- Whisper V2: adapter chạy openai-whisper offline, có hậu xử lý chống lặp và có output minh chứng trong output.
- PhoWhisper: ưu tiên model local dạng safetensors, thích hợp cho tiếng Việt nhưng hiện mới chứng minh chắc ở mức code và model local.
- WhisperX: thể hiện hướng nâng cấp mạnh nhất cho GUI timeline người nói, nhưng đang ở trạng thái kiểm chứng pipeline offline.

3.2. Giảm ảo giác trong ASR

Cherry Core dùng chiến lược giảm ảo giác nhiều lớp. Lớp thứ nhất là cấu hình giải mã bảo thủ trong WhisperV2Adapter; lớp thứ hai là Silero VAD loại bỏ khoảng lặng; lớp thứ ba là HallucinationFilter dùng Bag-of-Hallucinations và delooping. Cách tiếp cận này bám sát mục tiêu nghiên cứu trong kế hoạch NCKH: giảm bịa nội dung khi âm thanh kém chất lượng hoặc có vùng im lặng.

- Cấu hình condition_on_previous_text=False cắt chuỗi phụ thuộc lịch sử nhằm tránh lây lan lỗi.
- Ngưỡng compression_ratio_threshold, logprob_threshold, no_speech_threshold chặn segment đáng ngờ.
- Silero VAD có ngưỡng bảo thủ để ưu tiên không bỏ sót tiếng nói trong bối cảnh điều tra.
- Bag-of-Hallucinations mã hóa cả mẫu tiếng Việt lẫn tiếng Anh, phù hợp thực tế audio lẫn tạp âm hoặc slogan chèn.
- Delooping xử lý vòng lặp từ và cụm, điển hình trong các trường hợp Whisper tự lặp nhiều lần tên riêng hoặc filler.

3.3. Speaker diarization và căn chỉnh người nói

Nhánh diarization có tính chất nghiên cứu so sánh rõ rệt. Repo giữ song song Pyannote, SpeechBrain ECAPA-TDNN, EnhancedDiarizer dựa trên Resemblyzer và lớp VBx làm mượt nhãn speaker. Phần căn chỉnh AlignmentService đóng vai trò liên kết tất cả nhánh này về một transcript theo người nói.

- PyannoteAdapter hướng tới SOTA nhưng hiện vẫn chịu ràng buộc token hoặc license.
- SpeechBrainAdapter là lựa chọn offline rõ ràng hơn, có sẵn model cục bộ và kết nối trực tiếp với VBxRefiner.
- VBxRefiner dùng logic HMM hoặc Viterbi để giảm nhảy speaker không tự nhiên.
- AlignmentService dùng IntervalTree khi có thư viện, và linear fallback khi thiếu phụ thuộc.

3.4. Hậu hiệu chỉnh tiếng Việt

Lớp correction của Cherry Core đi theo cấu trúc nhiều tầng. Tầng một là VietnamesePhoneticCorrector sửa các lỗi đồng âm, lỗi phát âm và tên riêng dựa trên từ điển JSON. Tầng hai là ProtonXAdapter, một mô hình Seq2Seq cục bộ tối ưu cho correction văn bản dài. Tầng ba là CorrectionService kết hợp vocabulary injection và LLM cục bộ để sửa theo ngữ cảnh miền.

- Assets vocabulary cho thấy hệ được chuẩn bị cho các miền chuyên biệt như an ninh, pháp lý, khách sạn, tiếng lóng.
- PROOF_RESULTS.md chứng minh correction đã sửa được một số lỗi homophone và entity đáng chú ý.
- Thiết kế hiện tại phù hợp để GUI cho phép chạy correction tự động rồi cho điều tra viên duyệt lại thủ công.

3.5. Phân tích forensic bằng LLM

Mảng forensic LLM là nơi Cherry Core thể hiện tham vọng vượt khỏi hệ ASR thông thường. PromptManager dùng Jinja2 để ghép module 5W1H, SVA, SCAN, cảm xúc, nhạy cảm, suy luận vai trò và scenario YAML thành prompt điều tra. AnalysisService

hiện dùng LlamaCppAdapter làm engine mặc định, đồng thời repo đã chuẩn bị VLLMAdapter cho nhánh throughput cao hơn.

- Prompt deep_investigation.j2 ép đầu ra JSON-only, nhằm hướng tới báo cáo forensic có cấu trúc.
- Scenario YAML cho thấy hệ nhắm vào nhiều bối cảnh nghiệp vụ như ma túy, tình báo chung và lừa đảo công nghệ cao.
- Artifact output/..._analysis_report.json minh chứng lớp LLM đã được nối vào pipeline chạy thật.
- Tuy vậy, sample output hiện còn lưu cả raw_output với lỗi parse JSON, cho thấy structured generation chưa ổn định hoàn toàn.

3.6. Bằng chứng chạy thử và công cụ nghiên cứu

Ngoài mã nguồn, repo có hệ script và output khá dày. Đây là lớp evidence quan trọng để phân biệt một ý tưởng nghiên cứu với một thực thể kỹ thuật đã từng được kiểm chứng. scripts chứa benchmark, verify, setup và debug; tests chứa test logic prompt; output giữ transcript, segment JSON và report phân tích.

Chương 4. Các hướng nghiên cứu và nâng cấp đã xuất hiện trong repo

Khối docs của Cherry Core lưu nhiều báo cáo nghiên cứu có giá trị chiến lược. Những tài liệu này không nên bị hiểu nhầm là tính năng đã hoàn thiện; chúng là bản đồ nâng cấp của lõi hệ thống. Bảng dưới đây tổng hợp các nhánh nổi bật nhất cùng lợi ích, khoảng trống hiện tại và rủi ro triển khai.

Kỹ thuật đề xuất	Lợi ích kỳ vọng	Khoảng trống hiện tại	Rủi ro
Calm-Whisper	Giảm mạnh hallucination của Whisper bằng tinh chỉnh attention head gây lỗi, giữ WER gần như ổn định.	Cherry Core mới dừng ở cấu hình giải mã, VAD và BoH; chưa có quy trình fine-tune hoặc mô hình Calm-Whisper nội bộ.	Cần tài nguyên huấn luyện, tập dữ liệu nhiễu phù hợp và quy trình đánh giá riêng cho tiếng Việt.
Chain-of-Verification (CoVe)	Tăng độ tin cậy của LLM forensic bằng chu trình nháp - kiểm chứng - tổng hợp nhiều bước.	Repo hiện có phân tích LLM một lượt; chưa có service tách riêng các vòng xác minh hoặc scoring factual consistency.	Độ trễ tăng đáng kể, kéo theo nhu cầu cache và thiết kế job nền nếu muốn đưa vào GUI.
XGrammar / structured decoding	Nâng valid rate của JSON đầu ra lên mức gần tuyệt đối, phù hợp báo cáo chứng cứ có cấu trúc.	Cherry Core mới có GBNF cho llama.cpp và prompt JSON-only; chưa tích hợp XGrammar hoặc guided decoding thực sự.	Phụ thuộc backend LLM, có thể đòi hỏi chuyển hạ tầng sang vLLM hoặc một gateway riêng.
Semantic retrieval thay keyword matching	Lấy vocabulary và tri thức miền theo ngữ nghĩa thay vì trigger từ khóa thô, giúp correction bám sát ngữ cảnh hơn.	CorrectionService hiện đang chọn từ vựng bằng danh sách trigger thủ công; chưa có FAISS hoặc sentence-transformers.	Cần thêm pipeline index offline, đánh giá chi phí bộ nhớ và cơ chế cập nhật tri thức.
Pyannote Community-1 / 4.0	Cải thiện speaker assignment, speaker	Adapter đã nhắc tới Community-1 nhưng	Token, cache model, license và tương thích

	counting và hỗ trợ overlap tự nhiên hơn so với nhánh cũ.	repo chưa có benchmark DER hay artifact xác thực chính thức.	thiết bị là các điểm triển khai cần quản trị.
WhisperX migration	Hợp nhất ASR, alignment và diarization theo một pipeline end-to-end, phù hợp GUI timeline người nói.	Handoff cho thấy migration đang ở trạng thái kiểm chứng offline; pipeline mặc định của hệ chưa chuyển hẳn sang WhisperX.	Model nặng, phụ thuộc alignment/diarization assets và cần benchmark so với nhánh legacy.
vLLM qua WSL2 hoặc remote serving	Tăng throughput, hỗ trợ batching liên tục và mở đường cho guided decoding cấu trúc.	Adapter và tài liệu đã có, nhưng hiện trạng vẫn lấy llama.cpp làm fallback an toàn; chưa có mặt bằng triển khai ổn định cho GUI.	Windows native còn nhạy cảm; nếu tách thành service riêng thì phải bổ sung cơ chế health check và audit truy cập.

Tựu trung, định hướng nâng cấp của repo hội tụ vào ba trục: giảm ảo giác sâu hơn ở cả ASR và LLM; nâng chất lượng speaker diarization và structured output; chuẩn bị một hạ tầng LLM mạnh hơn để phục vụ GUI và xử lý nhiều hồ sơ. Đây là các trục hợp lý với đề bài NCKH ban đầu, nhưng cần được trình bày như roadmap có kiểm soát thay vì tính năng đã sẵn sàng đưa vào sản phẩm.

Chương 5. Kế hoạch hoàn thiện GUI web cho Cherry Core

GUI web được đề xuất như một lớp bao ngoài cho cherry_core, phục vụ điều tra viên hoặc nhà phân tích vận hành trên máy nội bộ. Mục tiêu của GUI không phải thay thế lõi xử lý hiện tại, mà biến lõi đó thành một công cụ tác nghiệp có thể nạp audio, theo dõi job, duyệt transcript, chạy phân tích LLM và xuất hồ sơ.

5.1. Nguyên tắc thiết kế

- Offline-first: toàn bộ audio, model, metadata và artifact nằm trong môi trường cục bộ sau khi cài đặt.
- Evidence-oriented: mọi chỉnh sửa, export và phân tích đều phải có audit trail.
- Human-in-the-loop: transcript và báo cáo LLM đều cho phép điều tra viên duyệt lại trước khi chốt.
- Frontend-neutral: đặc tả API và DTO độc lập với React, HTMX hay framework cụ thể.
- Không multi-tenant ở v1: tập trung một máy, một đơn vị vận hành, không thiết kế SSO hoặc cộng tác thời gian thực nhiều người.

5.2. Các màn hình bắt buộc

Màn hình	Mục tiêu	Dữ liệu vào	Dữ liệu ra	Ánh xạ lõi
Tổng quan hồ sơ	Theo dõi danh sách ca xử lý, trạng thái, mức độ ưu tiên và đường dẫn artifact.	Bộ lọc hồ sơ, trạng thái job, ngày xử lý, scenario phân tích.	Danh sách job, đường dẫn transcript, báo cáo, nhật ký xử lý.	Bọc file-based artifact trong output/ và metadata job cục bộ.
Tạo phiên xử lý audio	Nạp file âm thanh, chọn pipeline, chọn model và bật các tùy chọn correction/diarization.	Audio file, ASR engine, bật hoặc tắt diarization, bật correction, scenario.	Job mới với tham số chạy và đường dẫn input.	Gọi TranscribeAudioUseCase, SystemFactory, adapter ASR, diarizer và corrector.
Theo dõi job	Hiển thị tiến độ chạy, cảnh báo lỗi, thông	Job id, websocket	Tiến độ, trạng thái, lỗi, dấu	Wrapper quanh orchestration layer mới;

	tin model, thời gian xử lý và log.	stream hoặc polling.	mốc pipeline.	dùng log từ adapter và job metadata.
Transcript thô và transcript đã hiệu chỉnh	So sánh bản raw, corrected, đánh dấu chỗ bị nghi ngờ sai hoặc có thể chỉnh tay.	Transcript id, bộ lọc segment, thao tác chỉnh tay.	Transcript hiện hành, diff raw-corrected, lịch sử chỉnh sửa.	Dùng Transcript, CorrectionService, ProtonXAdapter, phonetic_corrector.
Timeline người nói	Xem segment theo thời gian, speaker block, word alignment và đoạn overlap nghi vấn.	Transcript segment, speaker segment, waveform hoặc timeline navigation.	Speaker-aware transcript và nhãn speaker theo đoạn.	Dùng SpeakerSegment, AlignmentService, adapter diarization và WhisperX.
Panel phân tích LLM	Chạy phân tích forensic, xem JSON hoặc raw output, entities, timeline và mức đe dọa.	Transcript đã duyệt, scenario, prompt profile.	Analysis report, extracted entities, hành vi, khuyến nghị.	Dùng AnalysisService, PromptManager, LlamaCppAdapter, vLLMAdapter.
Xuất báo cáo và chứng cứ	Đóng gói transcript, JSON báo cáo, nhật ký xử lý và bản DOCX nghiệp vụ.	Report id, định dạng xuất, danh sách artifact cần gộp.	Gói xuất bản TXT, JSON, DOCX kèm metadata.	Dùng OutputFormatter, serializer báo cáo, artifact writer mới.
Kiểm tra model và offline assets	Kiểm tra model có sẵn, dung lượng, trạng thái nạp, token yêu cầu và khả năng chạy offline.	Lệnh refresh inventory, yêu cầu preload model.	Danh sách model, kích thước, engine, trạng thái load, cảnh báo phụ thuộc.	Dùng core/config.py, SystemFactory, thư mục models/.

5.3. Luồng nghiệp vụ chuẩn

- Bước 1: người dùng tạo hồ sơ hoặc chọn hồ sơ sẵn có trên màn hình tổng quan.
- Bước 2: tải file audio và chọn engine ASR, bật correction hoặc diarization khi cần.
- Bước 3: hệ tạo JobStatus và khởi động pipeline chép lời.

- Bước 4: khi transcript hoàn thành, người dùng xem raw text, corrected text và timeline người nói.
- Bước 5: nếu có sai sót, người dùng chỉnh thủ công hoặc yêu cầu correction lại một phần transcript.
- Bước 6: transcript được duyệt đưa sang panel forensic để chạy prompt theo scenario.
- Bước 7: báo cáo phân tích được đọc, kiểm tra mức đe dọa, entity, timeline và khuyến nghị.
- Bước 8: người dùng xuất gói TXT, JSON, DOCX và hệ lưu audit trail cho toàn bộ thao tác.

5.4. Contract dữ liệu công khai

5.4.1. JobStatus

Trường	Kiểu	Ý nghĩa
job_id	string	Mã định danh job cục bộ, duy nhất theo phiên xử lý.
state	queued|running|completed|failed|cancelled	Trạng thái xử lý hiện thời.
stage	string	Tên bước đang chạy: upload, transcribe, diarize, correct, analyze, export.
progress	number	Tiến độ 0-100 cho UI hiển thị thanh trạng thái.
created_at	datetime	Thời điểm tạo job.
updated_at	datetime	Thời điểm cập nhật gần nhất.
error	string|null	Thông báo lỗi cuối nếu job thất bại.
artifacts	string[]	Danh sách đường dẫn artifact sinh ra.

5.4.2. TranscriptView

Trường	Kiểu	Ý nghĩa
transcript_id	string	Mã transcript gắn với một job.
job_id	string	Job nguồn đã sinh transcript.
language	string	Ngôn ngữ xử lý, mặc định vi.
raw_text	string	Bản transcript thô sau ASR.
corrected_text	string	Bản transcript sau correction hoặc chỉnh tay.
segments	TranscriptSegmentView[]	Danh sách segment theo thời gian.
speakers	SpeakerSegmentView[]	Danh sách segment người nói.
metadata	object	Thông tin model, flags anti-hallucination, output paths.

5.4.3. TranscriptSegmentView

Trường	Kiểu	Ý nghĩa
segment_id	string	Định danh segment.
start	number	Thời điểm bắt đầu theo giây.
end	number	Thời điểm kết thúc theo giây.
text	string	Nội dung segment.
speaker_id	string|null	Speaker gán cho segment, nếu có.
confidence	number|null	Độ tin cậy nếu backend cung cấp.
words	object[]	Danh sách từ có timestamp chi tiết.

5.4.4. SpeakerSegmentView

Trường	Kiểu	Ý nghĩa

speaker_id	string	Nhãn speaker chuẩn hóa, ví dụ SPEAKER_1.
start_time	number	Thời điểm bắt đầu.
end_time	number	Thời điểm kết thúc.
text	string	Văn bản gán cho speaker sau alignment.
words	object[]	Danh sách word-level alignment nếu có.

5.4.5. AnalysisReportView

Trường	Kiểu	Ý nghĩa
report_id	string	Mã báo cáo.
job_id	string	Liên kết job gốc.
scenario	string	Kịch bản điều tra được dùng.
summary	string	Tóm tắt cấp hồ sơ.
threat_level	string	Mức đe dọa chuẩn hóa.
classification	string	Phân loại hội thoại.
structured_data	object	Nội dung JSON có cấu trúc.
raw_output	string|null	Đầu ra LLM nguyên bản để đối chiếu.
validation_status	string	Trạng thái kiểm tra schema hoặc JSON.

5.4.6. ModelInfo

Trường	Kiểu	Ý nghĩa
model_id	string	Tên logic của model hoặc engine.
family	asr|diarization|correction|llm|vad	Nhóm chức năng.
path	string	Đường dẫn cục bộ tới model.

size_gb	number	Dung lượng ước tính.
offline_ready	boolean	Đã có file local đầy đủ hay chưa.
load_status	unloaded|loaded|error	Trạng thái nạp hiện tại.
notes	string	Ràng buộc đặc biệt như HF token, CUDA, WSL2.

5.4.7. AuditEntry

Trường	Kiểu	Ý nghĩa
entry_id	string	Mã nhật ký.
job_id	string|null	Job liên quan.
timestamp	datetime	Thời điểm phát sinh sự kiện.
actor	system|user	Nguồn phát sinh.
action	string	Hành động: upload, transcribe, manual_edit, export.
details	object	Thông tin bổ sung phục vụ truy vết.

5.5. Bộ endpoint v1

Method	Path	Mục đích	Đầu vào	Đầu ra	Lỗi thường gặp	Ánh xạ về Cherry Core
POST	/jobs/transcribe	Tạo job phiên âm mới từ audio đầu vào.	Multipart upload gồm file audio, engine ASR, cờ diarization, cờ correction và scenario.	Trả về JobStatus ban đầu cùng đường dẫn tới transcript khi hoàn thành.	400 file lỗi; 409 model chưa sẵn; 500 lỗi adapter hoặc thiếu	Wrapper mới quanh TranscribeAudioUseCase, SystemFactory, adapter ASR, VAD, diarizer và corrector.

					tài nguyên GPU/CPU.	
POST	/jobs/analyze	Khởi động pha phân tích forensic cho transcript đã chọn.	JSON gồm transcript_id, scenario, prompt_profile, cờ lưu raw output.	Trả về JobStatus của pha phân tích và report_id khi hoàn thành.	404 transcript không tồn tại; 409 transcript chưa sẵn sàng; 500 lỗi LLM.	Wrapper mới quanh AnalysisService, PromptManager, LlamaCppAdapter hoặc vLLMAdapter.
GET	/jobs/{id}	Lấy trạng thái và tiến độ thực thi của một job.	Path parameter id.	Một đối tượng JobStatus cập nhật mới nhất.	404 job không tồn tại.	Lớp read-model cục bộ bọc metadata job và log xử lý.
GET	/transcripts/{id}	Đọc transcript đầy đủ, gồm raw, corrected, segme	Path parameter id.	Đối tượng TranscriptView.	404 transcript không tồn tại; 409 artifact chưa sinh	Đọc từ Transcript, SpeakerSegment, AlignmentService, artifact trong output/.

		nt và speaker alignment.			xong.	
PATCH	/transcripts/{id}/corrections	Lưu chỉnh sửa thủ công hoặc yêu cầu correction lại một phần transcript.	JSON gồm segment cần sửa, kiểu sửa manual|phonetic|protonx|llm, và ghi chú điều tra viên.	Transcript sau cập nhật, kèm audit trail của lần chỉnh sửa.	400 payload sai; 404 transcript không có; 409 transcript đang bị khóa chỉnh sửa.	Wrapper mới quanh CorrectionService, VietnamesePhoneticCorrector, ProtonXAdapter và persistence layer cục bộ.
GET	/reports/{id}	Lấy báo cáo phân tích forensic có cấu trúc.	Path parameter id.	Đối tượng AnalysisReportView.	404 report không tồn tại.	Đọc artifact JSON trong output/ và ánh xạ về view model GUI.
POST	/reports/{id}/export	Xuất gói báo cáo ở	JSON gồm định dạng đích, danh sách artifact và metadata hồ sơ cần	Đường dẫn file xuất và checksum cục bộ.	404 report không tồn tại;	Wrapper mới quanh OutputFormatter, serializer báo cáo và export service

		dạng TXT, JSON, DOCX hoặc bundle hồ sơ.	chèn.		409 dữ liệu đầu vào thiếu; 500 lỗi export.	dùng filesystem nội bộ.
GET	/models	Liệt kê toàn bộ model offline hiện có trong hệ.	Không có.	Danh sách ModelInfo.	500 lỗi inventory hoặc quyền truy cập filesystem.	Đọc từ core/config.py, SystemFactory và thư mục models/.
POST	/models/{id}/load	Chủ động nạp model vào bộ nhớ trước khi chạy job.	Path parameter id, kèm tùy chọn target device nếu cần.	Đối tượng ModelInfo đã cập nhật load_status.	404 model không có; 409 model chưa offline-ready; 500 load thất bại.	Gọi load() trên adapter tương ứng thông qua SystemFactory.
GET	/audit/logs	Lấy nhật ký thao	Bộ lọc theo job, thời gian, actor hoặc action.	Danh sách AuditEntry.	500 lỗi đọc log hoặc	Thành phần mới bọc log file, metadata SQLite và action phát sinh từ

		tác phục vụ truy vết nghiệp vụ.			database cục bộ.	GUI.
WS	/jobs/{id}/stream	Đẩy tiến độ job, log và milestone pipeline theo thời gian thực.	Kết nối websocket theo job_id.	Sự kiện JobStatus, log event và trạng thái artifact mới.	404 job không tồn tại; 1011 lỗi backend stream.	Wrapper streaming quanh orchestration layer mới; không thay đổi adapter lõi.

5.6. Thiết kế lưu trữ và đóng gói offline

- Metadata job, transcript và audit trail dùng SQLite cục bộ để thuận tiện backup, tra cứu và triển khai một máy.
- Audio và artifact lớn tiếp tục lưu trên filesystem nội bộ theo thư mục hồ sơ; DB chỉ giữ metadata và checksum.
- Model inventory đọc trực tiếp từ models, tránh đồng bộ kép.
- Dịch vụ GUI có thể tách thành API wrapper, background worker và frontend bundle tĩnh; tuy nhiên v1 vẫn có thể chạy trong một tiến trình ứng dụng duy nhất.
- Mọi export DOCX phải đóng dấu nguồn transcript, model đã dùng, thời gian xử lý và cảnh báo về mức độ kiểm chứng của báo cáo LLM.

Chương 6. Hướng hoàn thiện, tiêu chí đánh giá và lộ trình nâng cấp

Cherry Core đã có một lõi kỹ thuật đáng kể, nhưng để trở thành hệ thống nghiên cứu - triển khai đủ vững cho môi trường điều tra thì cần thêm một pha ổn định hóa. Pha này không chỉ là bổ sung tính năng, mà còn là chuẩn hóa interface, khóa pipeline mặc định và làm rõ ranh giới giữa nhánh legacy với nhánh nghiên cứu.

6.1. Các điểm mạnh hiện tại

- Có nhiều lựa chọn ASR, diarization và LLM để so sánh trong cùng một lõi kiến trúc.
- Các thành phần offline asset đã hiện diện đầy đủ trong models, giảm phụ thuộc runtime download.
- Prompt forensic được xây tương đối bài bản, có scenario và module riêng cho nhiều khía cạnh phân tích.
- Repo giữ lại được output và báo cáo nghiên cứu, thuận tiện viết báo cáo NCKH và thiết kế GUI tiếp theo.

6.2. Khoảng trống kỹ thuật cần khóa trước khi bọc GUI

- Cần thống nhất interface LLM: GenerateStrategicReportUseCase gọi analyze() trong khi ILLMEngine công khai generate() và load().
- Cần đồng bộ entity hoặc domain với adapter: ví dụ PhoWhisperAdapter truyền thêm trường language không có trong dataclass hiện tại.
- Cần tách rõ entry point cũ và mới: cli.py còn dấu vết import hoặc service cũ, trong khi presentation/cli/main.py là nhánh hiện đại hơn.
- Structured JSON output chưa đủ ổn định; sample report còn phải lưu raw_output do parse lỗi.
- Chưa có persistence layer và job orchestration chính thức cho GUI; hiện repo vẫn thiên về script và CLI.

6.3. Tiêu chí đánh giá thành công cho giai đoạn hoàn thiện

- ASR: giữ hoặc cải thiện WER mục tiêu so với baseline hiện tại trên tập tiếng Việt thực tế.

- Hallucination: giảm rõ rệt segment lặp hoặc bịa ở vùng silence và có cơ chế flag transcript đáng ngờ.
- Diarization: có benchmark DER nhất quán cho ít nhất một pipeline mặc định.
- LLM: valid rate của JSON đạt mức chấp nhận được để GUI không phải dựa vào raw_output làm nguồn chính.
- Product: GUI MVP đủ các màn hình và endpoint đã nêu, hỗ trợ quy trình human-in-the-loop trọn vẹn.

6.4. Lộ trình triển khai đề xuất

- Giai đoạn 1 - Ổn định lõi: khóa interface, chọn pipeline mặc định, chuẩn hóa artifact và sửa các điểm mismatch port hoặc entity.
- Giai đoạn 2 - Tăng độ tin cậy: hoàn thiện structured output, bổ sung benchmark DER/WER hoặc hallucination và đóng gói audit trail.
- Giai đoạn 3 - Bọc API nội bộ: tạo persistence SQLite, background jobs và read-model cho transcript hoặc report.
- Giai đoạn 4 - GUI MVP: hiện thực các màn hình đã đặc tả, đưa transcript hoặc speaker timeline vào thao tác thực tế.
- Giai đoạn 5 - Nâng cấp nghiên cứu: CoVe, Calm-Whisper, XGrammar, semantic retrieval, vLLM serving ổn định.

Kết luận

Ở thời điểm khảo sát tháng 03/2026, cherry_core đã vượt qua mức một prototype đơn lẻ và hình thành một lõi kỹ thuật khá đầy đủ cho bài toán điều tra âm thanh tiếng Việt. Giá trị lớn nhất của repo nằm ở chỗ nó gom được nhiều hướng nghiên cứu về ASR, diarization, correction và forensic LLM trong cùng một khung kiến trúc. Giá trị còn lại cần hoàn thiện là biến khung đó thành sản phẩm tác nghiệp có giao diện, persistence và các chuẩn dữ liệu ổn định.

Phụ lục A. Bảng inventory mô-đun

Nhóm	Đường dẫn	Số file	Vai trò
core	core	20	Entity miền, port interface, cấu hình chung và dịch vụ alignment/output.
application	application	13	Application service và use case orchestration cho chép lời, correction, phân tích.
infrastructure	infrastructure	38	Adapter ASR, diarization, correction, LLM, VAD và factory nối lõi với model.
prompts	prompts	14	Template Jinja2, module prompt, grammar và scenario YAML cho phân tích forensic.
scripts	scripts	35	Tập script benchmark, setup model, verify pipeline và kiểm tra nhanh từng module.
tests	tests	8	Test cho prompt, deep analysis và khái quát hóa ngữ cảnh phân tích.
models	models	198	Kho model offline cho ASR, diarization, correction, VAD và LLM.
docs	docs	25	Tài liệu nghiên cứu, ghi chú chu trình nâng cấp

			và báo cáo đánh giá.
output	output	12	Transcript, diarization, JSON report và artifact chạy thử phục vụ minh chứng.

Phụ lục B. Bảng model offline

Model folder	Dung lượng	Số file	Tệp tiêu biểu	Trạng thái
phowhisper	5.75 GB	27	.gitattributes, added_tokens.json, config.json	offline-ready
phowhisper-full	3.43 MB	21	added_tokens.json, config.json, generation_config.json	offline-ready
phowhisper-safe	11.50 GB	30	.gitattributes, added_tokens.json, config.json	offline-ready
protonx	1.48 GB	7	config.json, dict.txt, generation_config.json	offline-ready
qwen3	4.68 GB	3	Qwen_Qwen3-8B-Q4_K_M.gguf, .gitignore, Qwen_Qwen3-8B-Q4_K_M.gguf.metadata	offline-ready
silero	2.22 MB	4	silero_vad.jit, utils_vad.py, __init__.py	offline-ready
speechbrain	85.01 MB	21	.gitattributes, classifier.ckpt, config.json	offline-ready
vistral	4.10 GB	3	vistral-7b-chat-	offline-ready

			Q4_K_M.gguf, .gitignore, vistral-7b-chat-Q4_K_M.gguf.metadata	
whisper-large-v2	23.00 GB	33	.gitattributes, added_tokens.json, config.json	offline-ready
whisper-large-v3	23.01 GB	43	.gitattributes, added_tokens.json, config.json	offline-ready
whisperx	2.88 GB	5	main, config.json, model.bin	offline-ready

Phụ lục C. Bảng script minh chứng

Script	Vai trò
scripts/__pycache__/generate_technical_doc_2026.cpython-311.pyc	Tác vụ tiện ích phục vụ nghiên cứu.
scripts/analyze_manual_transcript.py	Tác vụ tiện ích phục vụ nghiên cứu.
scripts/benchmark_full_file.py	Benchmark hoặc đo hiệu năng.
scripts/benchmark_llm.py	Benchmark hoặc đo hiệu năng.
scripts/build_comprehensive_vocab.py	Kiểm tra hoặc xây dựng thành phần correction hoặc vocabulary.
scripts/convert_phowhisper.py	Tác vụ tiện ích phục vụ nghiên cứu.
scripts/debug_alignment.py	Tác vụ tiện ích phục vụ nghiên cứu.
scripts/debug_whisperx_api.py	Tác vụ tiện ích phục vụ nghiên cứu.
scripts/download_single_sample.py	Chuẩn bị model, dữ liệu hoặc môi trường.
scripts/download_test_audio.py	Chuẩn bị model, dữ liệu hoặc môi trường.
scripts/download_whisper.py	Chuẩn bị model, dữ liệu hoặc môi trường.
scripts/full_diarization.py	Tác vụ tiện ích phục vụ nghiên cứu.

scripts/full_pipeline.py	Chạy pipeline xử lý âm thanh.
scripts/generate_online_sample.py	Tác vụ tiện ích phục vụ nghiên cứu.
scripts/generate_technical_doc_2026.py	Tác vụ tiện ích phục vụ nghiên cứu.
scripts/inspect_vivos.py	Tác vụ tiện ích phục vụ nghiên cứu.
scripts/prepare_benchmark_from_local.py	Benchmark hoặc đo hiệu năng.
scripts/quick_correction_check.py	Kiểm tra hoặc xây dựng thành phần correction hoặc vocabulary.
scripts/setup_models.py	Chuẩn bị model, dữ liệu hoặc môi trường.
scripts/setup_whisperx_offline.py	Chuẩn bị model, dữ liệu hoặc môi trường.
scripts/setup_wsl_vllm.ps1	Chuẩn bị model, dữ liệu hoặc môi trường.
scripts/step1_transcribe.py	Chạy pipeline xử lý âm thanh.
scripts/step2_diarize.py	Chạy pipeline xử lý âm thanh.
scripts/test_diarization.py	Kiểm thử nhanh hoặc xác minh module.
scripts/test_enhanced_diarization.py	Kiểm thử nhanh hoặc xác minh module.
scripts/test_grammar.py	Kiểm thử nhanh hoặc xác minh module.
scripts/test_hallucination_fix.py	Kiểm thử nhanh hoặc xác minh module.
scripts/test_phonetic.py	Kiểm thử nhanh hoặc xác minh module.
scripts/test_user_audio.py	Kiểm thử nhanh hoặc xác minh module.
scripts/test_vocab_loading.py	Kiểm thử nhanh hoặc xác minh module.
scripts/test_vocab_retrieval.py	Kiểm thử nhanh hoặc xác minh module.
scripts/verify_correction_standalone.py	Kiểm thử nhanh hoặc xác minh module.
scripts/verify_whisperx.py	Kiểm thử nhanh hoặc xác minh module.
scripts/wsl_setup.sh	Chuẩn bị model, dữ liệu hoặc môi trường.
scripts/wsl_verify.sh	Kiểm thử nhanh hoặc xác minh module.

Phụ lục D. Bảng test minh chứng

Test file	Vai trò
tests/__pycache__/test_deep_analysis.cpython-311-pytest-7.4.3.pyc	Kiểm tra phân tích forensic bằng LLM.
tests/__pycache__/test_deep_prompt.cpython-311-pytest-7.4.3.pyc	Xác minh prompt render hoặc logic prompt.
tests/__pycache__/test_generalization.cpython-311-pytest-7.4.3.pyc	Kiểm tra khả năng khái quát hóa hoặc bám scenario.
tests/__pycache__/test_prompt_renderer.cpython-311-pytest-7.4.3.pyc	Xác minh prompt render hoặc logic prompt.
tests/test_deep_analysis.py	Kiểm tra phân tích forensic bằng LLM.
tests/test_deep_prompt.py	Xác minh prompt render hoặc logic prompt.
tests/test_generalization.py	Kiểm tra khả năng khái quát hóa hoặc bám scenario.
tests/test_prompt_renderer.py	Xác minh prompt render hoặc logic prompt.

Phụ lục E. Bảng output minh chứng

Artifact	Vai trò
output/sample/Result.json	Artifact JSON phục vụ báo cáo, segment hoặc phân tích.
output/sample/Result.txt	Bản text phục vụ kiểm tra tay hoặc trình bày.
output/test_audio_diarized_final.txt	Kết quả gán speaker hoặc transcript theo người nói.
output/test_audio_segments.json	Artifact JSON phục vụ báo cáo, segment hoặc phân tích.
output/Tiếp nhận yêu cầu đặt phòng của khách lẻ qua điện thoại_analysis_report.json	Artifact JSON phục vụ báo cáo, segment hoặc phân tích.
output/Tiếp nhận yêu cầu đặt phòng của khách lẻ	Artifact JSON phục vụ báo cáo, segment hoặc

qua điện thoại_diarized.json	phân tích.
output/Tiếp nhận yêu cầu đặt phòng của khách lẻ qua điện thoại_diarized.txt	Kết quả gán speaker hoặc transcript theo người nói.
output/Tiếp nhận yêu cầu đặt phòng của khách lẻ qua điện thoại_transcript_corrected.txt	Transcript raw hoặc corrected.
output/Tiếp nhận yêu cầu đặt phòng của khách lẻ qua điện thoại_transcript_raw.txt	Transcript raw hoặc corrected.
output/transcript_v2_tuned.txt	Transcript raw hoặc corrected.
output/transcript_v3_tuned.txt	Transcript raw hoặc corrected.
output/transcript_whisperv3.txt	Transcript raw hoặc corrected.

Phụ lục F. Mapping implemented vs proposed

Hạng mục	Nhóm	Nhãn	Minh chứng
Whisper V2 với cấu hình chống ảo giác	Đã triển khai	đã có output minh chứng	infrastructure/adapters/asr/whisperv2_adapter.py; PROOF_RESULTS.md
PhoWhisper cho ASR tiếng Việt	Đã triển khai	đã có code	infrastructure/adapters/asr/phowhisper_adapter.py; models/phowhisper-safe/
WhisperX end-to-end (ASR + alignment + diarization)	Đã triển khai	đã có script kiểm thử	infrastructure/adapters/asr/whisperx_adapter.py; scripts/setup_whisperx_offline.py
Decoding thresholds giảm hallucination	Đã triển khai	đã có code	infrastructure/adapters/asr/whisperv2_adapter.py
Silero VAD tiền xử lý	Đã triển khai	đã có code	infrastructure/adapters/vad/silero_adapter.py; models/silero/
Bag-of-	Đã triển	đã có code	infrastructure/adapters/asr/hallucination_filter.py

Hallucinations tiếng Việt và tiếng Anh	khai		
Delooping hậu xử lý	Đã triển khai	đã có code	infrastructure/adapters/asr/hallucination_filter.py; infrastructure/adapters/asr/whisperv2_adapter.py
Pyannote diarization	Đã triển khai	mới ở mức adapter/probe	infrastructure/adapters/diarization/pyannote_adapter.py; docs/CYCLE_2_RESEARCH_DIARIZATION.md
SpeechBrain ECAPA-TDNN diarization	Đã triển khai	đã có code	infrastructure/adapters/diarization/speechbrain_adapter.py; models/speechbrain/
VBx/Viterbi resegmentation	Đã triển khai	đã có code	infrastructure/adapters/diarization/vbx_refiner.py; infrastructure/adapters/diarization/speechbrain_adapter.py
Word-speaker alignment bằng IntervalTree	Đã triển khai	đã có code	core/services/alignment_service.py
Rule-based phonetic correction	Đã triển khai	đã có code	application/services/phonetic_corrector.py; assets/vocab/vietnamese_phonetic_errors.json
ProtonX Seq2Seq correction	Đã triển khai	đã có output minh chứng	infrastructure/adapters/correction/protonx_adapter.py; PROOF_RESULTS.md
LLM contextual correction	Đã triển khai	đã có code	application/services/correction_service.py; application/services/phonetic_corrector.py
Prompt modules và scenario YAML	Đã triển khai	đã có script kiểm thử	application/services/prompt_manager.py; tests/test_prompt_renderer.py
JSON-only prompting và	Đã triển khai	đã có code	prompts/templates/deep_investigation.j2; prompts/grammars/json_schema.gbnf

GBNF			
LLM offline qua llama.cpp	Đã triển khai	đã có output minh chứng	infrastructure/adapters/llm/llamacpp_adapter.py; output/Tiếp nhận yêu cầu đặt phòng của khách lẻ qua điện thoại_analysis_report.json
vLLM fallback qua WSL2/probe	Đã triển khai	đã có script kiểm thử	infrastructure/adapters/llm/vllm_adapter.py; scripts/setup_wsl_vllm.ps1
Calm-Whisper	Đề xuất/Nâng cấp	mới ở mức adapter/probe	docs/DEEP_RESEARCH_UPGRADE_2026.md; docs/UPGRADE_RESEARCH_2026.md
Chain-of-Verification (CoVe)	Đề xuất/Nâng cấp	mới ở mức adapter/probe	docs/DEEP_RESEARCH_UPGRADE_2026.md; docs/ADVANCED_RESEARCH_PROPOSAL.md
XGrammar / structured decoding	Đề xuất/Nâng cấp	mới ở mức adapter/probe	docs/DEEP_RESEARCH_UPGRADE_2026.md; docs/CYCLE_4_DESIGN_VLLM.md
Semantic retrieval thay keyword matching	Đề xuất/Nâng cấp	mới ở mức adapter/probe	docs/DEEP_RESEARCH_UPGRADE_2026.md
Pyannote Community-1 / 4.0	Đề xuất/Nâng cấp	mới ở mức adapter/probe	infrastructure/adapters/diarization/pyannote_adapter.py; docs/SOTA_DIARIZATION_RESEARCH.md
WhisperX migration	Đề xuất/Nâng cấp	mới ở mức adapter/probe	AGENT_HANDOFF.md; infrastructure/adapters/asr/whisperx_adapter.py
vLLM qua WSL2 hoặc remote serving	Đề xuất/Nâng cấp	mới ở mức adapter/probe	docs/CYCLE_4_DESIGN_VLLM.md; docs/CYCLE_4_STATUS.md

Phụ lục G. Glossary thuật ngữ

Thuật ngữ	Giải nghĩa
Hallucination	Hiện tượng mô hình sinh ra văn bản không tồn tại trong âm thanh hoặc không được chứng minh bởi dữ liệu đầu vào.
Anti-hallucination	Tập hợp kỹ thuật cấu hình, tiền xử lý và hậu xử lý để giảm nội dung bịa.
Diarization	Quá trình xác định ai nói câu nào theo trục thời gian.
Alignment	Quá trình gắn mốc thời gian cho từ hoặc câu để liên kết ASR với speaker segment.
Offline-first	Nguyên tắc thiết kế cho phép hệ thống tiếp tục hoạt động sau khi đã cài đặt model cục bộ, không phụ thuộc internet.
Forensic report	Báo cáo trích xuất thông tin điều tra, có cấu trúc, phục vụ phân tích nghiệp vụ.
Artifact	Tệp đầu ra, log, transcript, báo cáo, hoặc metadata sinh ra trong một job xử lý.

Phụ lục H. Tài liệu tham khảo

[1] Kế hoạch thực hiện nhiệm vụ nghiên cứu khoa học học viên năm học 2025-2026, file nguồn do người dùng cung cấp.

[2] E:/research/Cherry2/cherry_core/docs/DEEP_RESEARCH_UPGRADE_2026.md - báo cáo nghiên cứu chuyên sâu về anti-hallucination, diarization và structured output.

[3] E:/research/Cherry2/cherry_core/docs/SOTA_DIARIZATION_RESEARCH.md - khảo sát hướng diarization SOTA và VBx resegmentation.

[4] E:/research/Cherry2/cherry_core/docs/SCIENTIFIC_PROMPT_RESEARCH.md - khung prompt forensic dựa trên SVA/SCAN/ACH.

[5] E:/research/Cherry2/cherry_core/docs/WHISPER_MODEL_SELECTION.md - cơ sở chọn Whisper V2 làm baseline ít hallucination hơn V3.

[6] E:/research/Cherry2/cherry_core/docs/CYCLE_4_DESIGN_VLLM.md và CYCLE_4_STATUS.md - thiết kế backend LLM hiệu năng cao và cơ chế fallback.

[7] E:/research/Cherry2/cherry_core/PROOF_RESULTS.md - bằng chứng chạy thử correction, anti-hallucination và GPU.

Ghi chú nguồn từ kế hoạch NCKH

Các trích dẫn học thuật gốc xuất hiện trong kế hoạch NCKH của đề tài được giữ lại như nguồn định hướng cho phần nghiên cứu và nâng cấp của Cherry Core.

[Tên học viên 1] - [Lớp/Khoá]

[1] Le D.T. et al. (2024). "PhoWhisper: Automatic Speech Recognition for Vietnamese." ICLR 2024 Tiny Papers. arXiv:2406.02555

[2] Baranski M. et al. (2025). "Investigation of Whisper ASR Hallucinations Induced by Non-Speech Audio." arXiv:2501.11378

[3] Wang J. et al. (2025). "Calm-Whisper: Reduce Whisper Hallucination By Calming Crazy Heads Down." Interspeech 2025. arXiv:2505.12969

[4] Tran et al. (2025). "VietASR: Achieving Industry-level Vietnamese ASR with 50-hour labeled data." arXiv:2505.21527

[5] Bredin H. et al. (2023). "Pyannote.audio 2.1 Speaker Diarization Pipeline." ICASSP 2023. arXiv:2212.02060

[6] Vo T.H. et al. (2025). "Speaker Diarization for Low-Resource Languages Through Wav2Vec Fine-Tuning." arXiv:2504.18582

[7] Dhuliawala S. et al. (2024). "Chain-of-Verification Reduces Hallucination in Large Language Models." ACL 2024 Findings. arXiv:2309.11495

[8] Dong M. et al. (2024). "XGrammar: Flexible and Efficient Structured Generation Engine for LLMs."

[9] Nguyen L.T. et al. (2021). "BARTpho: Pre-trained Sequence-to-Sequence Models for Vietnamese." arXiv:2109.09701